SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-015: Student Records Export to CSV

Field	Details
Change Request ID	CR-015
Change Request Title	Student Records Export to CSV
Student Name	A.Paheershana
Student ID	JF/ICT/24/04
Implementation Status	Completed
Date	15 October 2026

1. Change Request Summary

1.1 Business Requirement

The Student Management System required a facility that allows administrators to export student records into a CSV (Comma-Separated Values) file for reporting, record-keeping, and external data processing.

Previously, administrators could view, search, add, edit, and delete student records through the Student Management System interface. However, a dedicated CSV export facility was required to allow administrators to download student information for administrative purposes.

The new feature introduces an Export CSV button to the administrator's student management dashboard. When the administrator clicks this button, the system generates and downloads a file named students.csv.

The exported file contains the following information:

- Student ID
- Registration Number
- First Name

- Last Name
- Email
- Phone Number

The implementation uses the student records already retrieved from the backend API. Therefore, no additional database tables, database columns, or backend API endpoints are required.

The main objective is to improve administrative efficiency and provide a convenient way to maintain and process student records while preserving existing authentication, authorization, and student management functionality.

.

1.2 Existing Behaviour

Before implementation:

- Administrators could log in using their existing credentials.
- Administrators could view student records on the dashboard.
- The system displayed student registration numbers, names, email addresses, phone numbers, and last login information where available.
- Administrators could search student records using the existing search functionality.
- Administrators could add new student records.
- Administrators could edit existing student records.
- Administrators could delete student records.
- Student data was retrieved through the existing GET /api/students/ endpoint.
- The backend restricted access to student management operations using authentication and role-based authorization.
- A dedicated facility for downloading student records as a CSV file was not available.

1.3 Requested Change

The requested change was to introduce a CSV export facility into the administrator's student management dashboard.

The following functionality was required:

- Add an Export CSV button to the student management interface.
- Allow administrators to export student records into a CSV file.
- Include Student ID, Registration Number, First Name, Last Name, Email, and Phone Number in the exported file.
- Use the filename students.csv.
- Include meaningful column headers.
- Support exporting multiple student records.
- Generate a valid CSV file that can be opened using spreadsheet applications.
- Handle empty student lists appropriately.
- Handle special characters and quotation marks in student information.
- Prevent the export action while student data is loading or when the data request has failed.
- Preserve the existing Add, Edit, Delete, and Search functionality.
- Preserve existing authentication and administrator authorization.
- Avoid unnecessary backend and database modifications.
- Keep the interface responsive and user-friendly.

2. Implementation

2.1 Implementation Summary

CR-015 was implemented by extending the existing Student Management System frontend.

The `DashboardPage.jsx` component was updated to include a CSV export function and an Export CSV button in the student management dashboard.

The export function uses the student records already stored in the frontend state after they have been retrieved through the existing API request.

The implementation generates a CSV file containing the required student information. Each record is converted into a CSV row, and the column names are included as the first row.

The implementation also handles CSV escaping by enclosing values in quotation marks and escaping embedded quotation marks. This helps preserve student information containing commas, quotation marks, or other special characters.

A UTF-8 byte-order mark is included to improve compatibility with spreadsheet applications when displaying text.

The generated CSV content is placed into a browser `Blob`, and a temporary download link is used to download the file as `students.csv`.

The dashboard stylesheet was updated to provide appropriate button styling, hover effects, disabled states, and responsive layout behaviour.

The existing backend API already provides the required student fields. Therefore, the existing controller, model, routes, and database structure were reused without modification.

2.2 Affected Files

File	Change	Purpose
frontend/src/pages/DashboardPage.jsx	Modified	Implements CSV generation, file download, and the Export CSV button.
frontend/src/styles/dashboard.css	Modified	Adds styling for the Export CSV button and responsive layout.
backend/src/controllers/studentController.js	Reused	Existing getStudents() controller provides student records.
backend/src/models/studentModel.js	Reused	Existing getAllStudents() query retrieves the required student fields.
backend/src/routes/studentRoutes.js	Reused	Existing administrator-only route protects access to all student records.
backend/src/services	No change required	No additional backend service is required for client-side CSV generation.
Database schema	No change required	Existing student data is reused.
Backend API	No new endpoint	Existing GET /api/students/ endpoint is reused.

2.3 Technical Changes

2.3.1 DashboardPage.jsx Modification

The DashboardPage.jsx component was modified to introduce the CSV export feature.

The component already retrieves student records through the existing API request:

GET /api/students/

The retrieved records are stored in the students state variable.

The new export functionality uses this state to generate the CSV file without making an additional API request.

The Export CSV button is placed in the student management heading section alongside the existing Add Student button.

The export functionality includes:

- Reading the available student records.
- Defining the CSV column headers.
- Selecting the required fields from each student record.
- Converting student records into CSV rows.
- Escaping quotation marks and special characters.
- Generating CSV content using the required column order.
- Creating a browser Blob.
- Creating a temporary download link.
- Downloading the file as students.csv.
- Releasing the temporary object URL after the download is initiated.

The CSV file contains the following columns:

1. Student ID
2. Registration Number
3. First Name
4. Last Name
5. Email

6. Phone Number

The export functionality does not modify the original student records.

2.3.2 CSV Generation Logic

The export function creates a header row and converts each student object into a row containing the required fields.

Illustrative implementation logic:

```
const headers = [
  "Student ID",
  "Registration Number",
  "First Name",
  "Last Name",
  "Email",
  "Phone Number"
];


const rows = students.map((student) => [
  student.student_id,
  student.registration_no,
  student.first_name,
  student.last_name,
  student.email,
  student.phone_number
]);
```

The header row and student rows are combined to create the CSV content.

Each value is escaped before being joined with commas. This helps prevent commas and quotation marks inside a student's information from incorrectly splitting the CSV columns.

The CSV rows are separated using carriage return and line feed characters to improve compatibility with common spreadsheet applications.

2.3.3 File Download Implementation

The generated CSV content is converted into a browser Blob with the MIME type:

text/csv;charset=utf-8;

A temporary object URL is created from the Blob.

The browser download mechanism uses the filename:

students.csv

After the download is initiated, the temporary link is removed and the object URL is released.

This approach allows CSV files to be generated directly in the browser without requiring a separate server-side file-generation endpoint.

2.3.4 Export CSV Button Modification

An Export CSV button was added to the student management heading section.

The button is placed alongside the existing Add Student button.

The button provides the following behaviour:

- Starts the CSV export operation when clicked.
- Is disabled while student records are loading.
- Is disabled when the student data request has failed.
- Provides visual feedback through hover and disabled states.
- Does not interfere with existing student management actions.

If no student records are available, the application displays an appropriate message instead of downloading an empty data file.

The existing Add Student button and its functionality remain unchanged.

2.3.5 Dashboard CSS Modification

The dashboard.css stylesheet was updated to support the new Export CSV button.

The styling includes:

- Button padding and border styling.
- Consistent font size and font weight.
- Appropriate text and background colours.
- Hover effects.
- Keyboard focus indication.
- Disabled button styling.
- Responsive layout behaviour for smaller screens.

The Export CSV button uses a visual style consistent with the existing dashboard interface.

The .students-heading-actions class arranges the Export CSV and Add Student buttons together with appropriate spacing.

A responsive media query adjusts the layout on smaller screens to prevent the buttons from becoming difficult to use.

2.3.6 Backend Controller Reuse

The existing backend/src/controllers/studentController.js file contains the getStudents() controller.

This controller retrieves the student records through the existing getAllStudents() model function and returns them as JSON.

The controller already provides the information needed by the frontend CSV export function.

Therefore, no new CSV export controller or backend endpoint was required.

The existing controller continues to handle database errors and return an appropriate response.

2.3.7 Student Model Reuse

The existing backend/src/models/studentModel.js file contains the getAllStudents() function.

The database query retrieves the following information:

- student_id
- registration_no
- first_name
- last_name
- email
- phone_number

It also retrieves additional fields such as user_id, created_at, updated_at, and last_login for existing application functionality.

The CSV export selects only the six fields required by CR-015.

No changes to the SQL query or database model are necessary.

2.3.8 Route and Authorization Preservation

The existing backend/src/routes/studentRoutes.js file defines the administrator-only route for retrieving all student records.

The route uses the existing authentication and role authorization middleware:

```
router.get(
  "/",
  authenticate,
  authorizeRoles("ADMIN"),
  getStudents
);
```

The authenticate middleware verifies the user's authentication, while authorizeRoles("ADMIN") restricts access to administrators.

The CSV export feature uses the records returned by this existing protected endpoint.

No new public endpoint was introduced, and no changes to the existing authorization rules were required.

2.3.9 Database Modification

No database modifications were required for CR-015.

The existing students table already contains the student information required for CSV export.

The implementation does not create new tables, add columns, change existing data types, or modify database relationships.

The feature reads existing student records and generates a downloadable file in the browser.

This reduces implementation complexity and avoids unnecessary database migration risks.

2.3.10 Error Handling and Data Management

The implementation handles the following conditions:

- Student records are still loading.
- The student data request fails.
- No student records are available.
- Student fields contain commas or quotation marks.
- Student fields contain empty or null values.
- The browser creates the CSV file and initiates the download.

The export button remains disabled while data is loading or when the existing data request has failed.

If no records are available, an appropriate message is displayed.

The export function uses CSV escaping to preserve field boundaries when values contain commas or quotation marks.

The implementation does not change, delete, or update student information in the database.

2.3.11 Existing Functionality Preservation

CR-015 was implemented as a frontend enhancement without replacing the existing student management operations.

The following existing functionality remains available:

- Administrator login and authentication.
- Student record retrieval.
- Student search.
- Add Student.
- Edit Student.
- Delete Student.
- Last Login information display.
- Existing API error handling.
- Role-based access control.

The new export function is independent of the existing Add, Edit, and Delete operations.

2.4 Implementation Logic

The implementation follows these steps:

1. The administrator opens the Student Management System.
2. The administrator logs in using valid credentials.
3. The dashboard loads the student records.
4. The frontend requests student information through GET /api/students/.
5. The backend authenticates the request.
6. The backend verifies that the user has the ADMIN role.
7. The backend retrieves the student records from the database.
8. The controller returns the student records as JSON.
9. The frontend stores the records in the students state.
10. The administrator clicks the Export CSV button.
11. The frontend checks whether student data is available.
12. The application defines the required CSV headers.
13. The application selects the required fields from each student record.
14. Each field is escaped to preserve valid CSV formatting.
15. The header row and student rows are combined.
16. The application creates a CSV Blob.
17. A temporary download URL is generated.
18. The browser initiates the download.
19. The file is downloaded as students.csv.
20. The temporary download link and object URL are cleaned up.
21. The administrator can open the file using a compatible spreadsheet or text application.
22. Existing student management functionality remains available.

2.5 Error and Data Handling

The implementation follows these data-handling rules:

- **Student records available:** Generate and download the CSV file.
- **Student list empty:** Display an appropriate message.
- **Data loading:** Disable the export button until loading is complete.
- **API request failure:** Disable the export button and preserve the existing error handling.
- **Empty field:** Represent the missing value safely in the CSV output.
- **Comma in a field:** Escape the field correctly.
- **Quotation mark in a field:** Escape embedded quotation marks.
- **Multiple student records:** Generate one CSV row for each available record.
- **Successful export:** Download the file using the required filename.
- **Authentication failure:** Preserve the existing authentication handling.
- **Unauthorized request:** Preserve the backend's administrator-only authorization.
- **Database error:** Use the existing backend error handling.
- **Existing student records:** Do not modify the database records during export.

The export operation is read-only and does not change the existing student management data.

3. Testing & Verification

3.1 Testing Approach

The implementation was tested to verify that administrators can export student records into a valid CSV file and that the existing Student Management System functionality remains operational.

Testing focused on CSV file generation, the download filename, column headers, student data accuracy, multiple records, empty data handling, CSV escaping, button behaviour, and regression testing.

Testing Types:

- Functional testing.
- CSV export testing.
- Data accuracy testing.
- Frontend validation testing.
- Error-handling testing.
- UI testing.
- Responsive interface testing.
- Authorization regression testing.
- Existing functionality regression testing.

Environment: Development

Browser: Google Chrome / Microsoft Edge

Frontend: React and Vite

Backend: Node.js and Express.js

Database: MySQL

API: Existing GET /api/students/ endpoint

Test Scope: Student Records Export to CSV and existing student management functionality.

3.2 Test Cases

Copy the table below and paste it directly into Microsoft Word.

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Open the administrator dashboard.	The dashboard loads correctly.	The dashboard loaded correctly.	Pass
TC-02	Verify that student records are loaded.	Student records are displayed.	Student records were displayed.	Pass
TC-03	Verify the Export CSV button.	The button is displayed in the student management section.	The button was displayed correctly.	Pass
TC-04	Click the Export CSV button when records are available.	A CSV file is downloaded.	The CSV file was downloaded.	Pass
TC-05	Check the downloaded filename.	The filename is students.csv.	The filename was correct.	Pass
TC-06	Open the downloaded CSV file.	The file opens in a compatible application.	The file opened correctly.	Pass
TC-07	Verify the CSV column headers.	All six required headers are present.	All required headers were present.	Pass
TC-08	Verify the Student ID values.	The correct student IDs are exported.	Student IDs were exported correctly.	Pass
TC-09	Verify the Registration Number values.	The correct registration numbers are exported.	Registration numbers were exported correctly.	Pass
TC-10	Verify the First Name and Last Name values.	Student names are exported correctly.	Student names were exported correctly.	Pass
TC-11	Verify the Email values.	Student email addresses are exported correctly.	Email addresses were exported correctly.	Pass

TC-12	Verify the Phone Number values.	Student phone numbers are exported correctly.	Phone numbers were exported correctly.	Pass
TC-13	Export multiple student records.	Each available record appears in a separate CSV row.	Multiple records were exported correctly.	Pass
TC-14	Export when no student records are available.	An appropriate message is displayed.	The expected message was displayed.	Pass
TC-15	Verify export button behaviour during loading.	The button remains disabled while data is loading.	The button remained disabled.	Pass
TC-16	Verify export button behaviour after an API error.	The button remains disabled when the data request fails.	The button remained disabled.	Pass
TC-17	Verify CSV values containing commas or quotation marks.	Special characters are handled without corrupting CSV columns.	CSV escaping worked correctly.	Pass
TC-18	Verify empty or null field values.	Empty values do not break the CSV structure.	Empty values were handled correctly.	Pass
TC-19	Verify Export CSV button styling.	The button displays the expected styling and hover behaviour.	Styling and hover behaviour worked correctly.	Pass
TC-20	Verify the dashboard on a smaller screen.	The buttons remain usable on smaller screens.	The responsive layout worked correctly.	Pass
TC-21	Verify student search functionality.	Existing search functionality continues to work.	Search functionality worked correctly.	Pass
TC-22	Verify Add Student functionality.	Administrators can add student records.	Add Student functionality remained operational.	Pass
TC-23	Verify Edit Student functionality.	Administrators can update student records.	Edit Student functionality remained operational.	Pass

TC-24	Verify Delete Student functionality.	Administrators can delete student records.	Delete Student functionality remained operational.	Pass
TC-25	Verify administrator authorization.	The student list endpoint remains restricted to administrators.	Existing administrator authorization remained operational.	Pass
TC-26	Verify database records after export.	Exporting does not modify student records.	Student records remained unchanged.	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	26
Passed	26
Failed	0
Pass Rate	100%

Overall Test Result: Pass

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	The Export CSV button is available on the administrator dashboard.	UI inspection	Met
AC-002	The administrator can download student records as a CSV file.	Functional testing	Met
AC-003	The exported file uses the filename students.csv.	Download verification	Met
AC-004	The CSV file contains meaningful column headers.	File inspection	Met
AC-005	The CSV contains Student ID, Registration Number, First Name, Last Name, Email, and Phone Number.	Data verification	Met
AC-006	Multiple student records can be exported.	CSV inspection	Met
AC-007	Empty student lists are handled appropriately.	Empty-data testing	Met
AC-008	CSV values containing commas and quotation marks are handled correctly.	CSV formatting testing	Met
AC-009	The export button is disabled while data is loading or the data request has failed.	UI and error-handling testing	Met
AC-010	The existing student data API can be reused without adding a new endpoint.	API verification	Met
AC-011	Existing authentication and administrator authorization are preserved.	Authorization testing	Met
AC-012	The existing Add Student functionality remains operational.	Regression testing	Met
AC-013	The existing Edit Student functionality remains operational.	Regression testing	Met
AC-014	The existing Delete Student functionality remains operational.	Regression testing	Met

AC-015	The existing student search functionality remains operational.	Regression testing	Met
AC-016	No database schema modification is required.	Database and code inspection	Met
AC-017	The CSV export does not modify student records.	Database verification	Met
AC-018	The export interface remains usable on smaller screens.	Responsive UI testing	Met

5. Evidence

Evidence 01 – Administrator Dashboard

Related Acceptance Criteria: AC-001

Description: Insert a screenshot of the administrator dashboard showing the student management section and the Export CSV button.

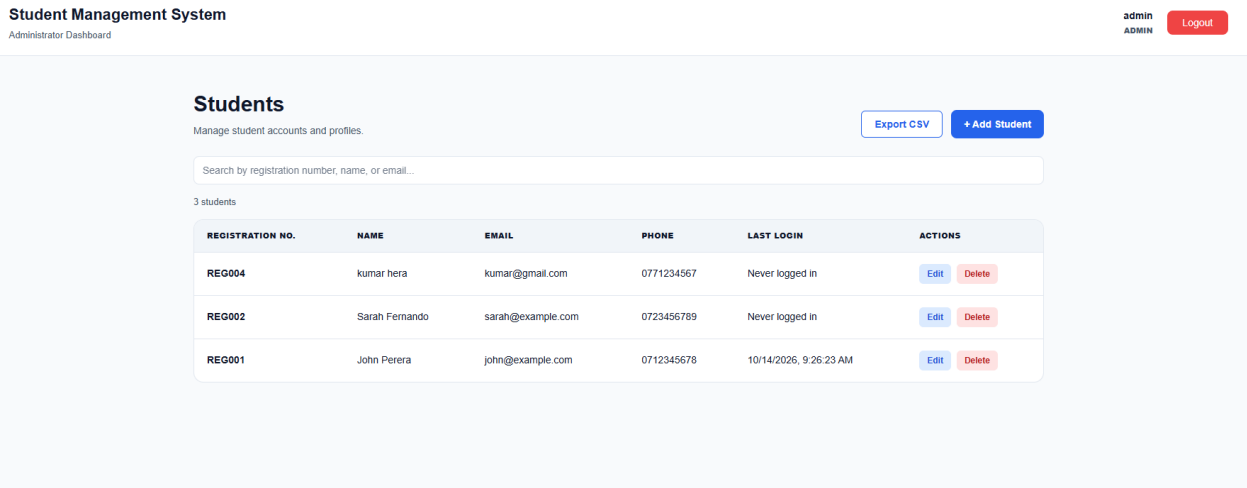


Figure 1: Student management dashboard with Export CSV button

Evidence 02 – CSV File Download

Related Acceptance Criteria: AC-002, AC-003

Description: Insert a screenshot showing the downloaded students.csv file in the browser's download list or the computer's Downloads folder.

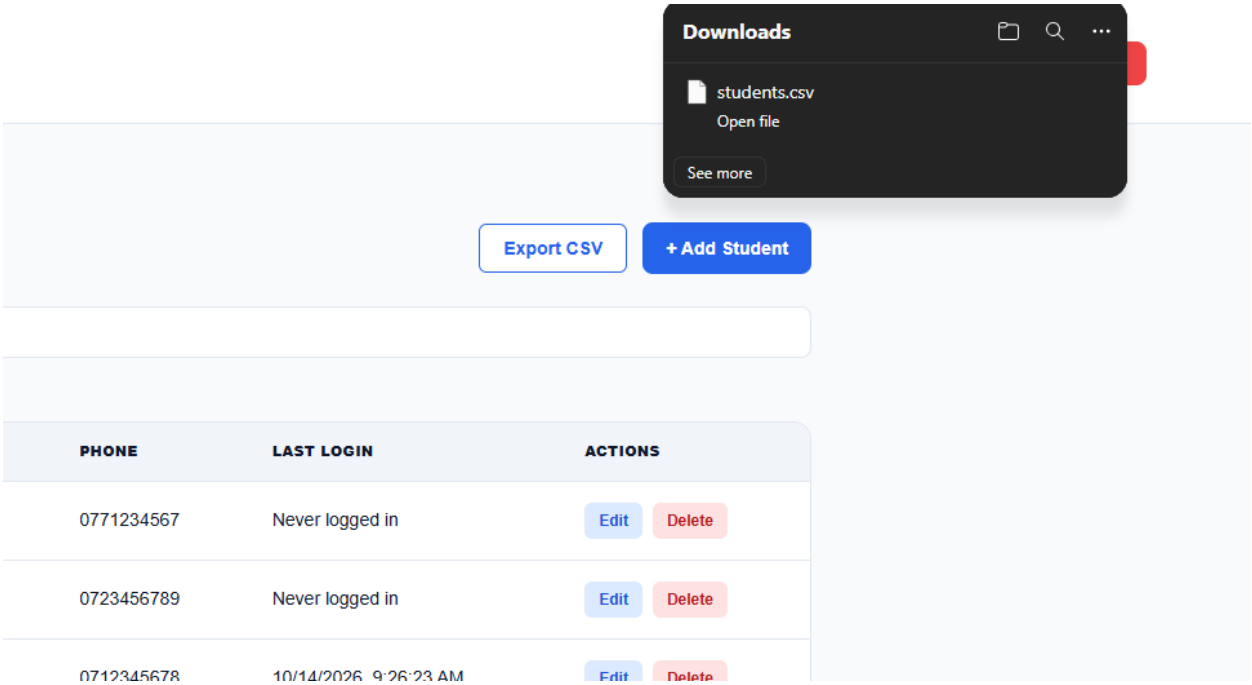


Figure 2: Successfully downloaded students.csv file

Evidence 03 – CSV Column Headers

Related Acceptance Criteria: AC-004, AC-005

Description: Insert a screenshot of the opened CSV file showing the six required column headers: Student ID, Registration Number, First Name, Last Name, Email, and Phone Number.

Student ID	Registration Number	First Name	Last Name	Email	Phone Number
6	REG004	kumar	hera	kumar@gmail.com	771234567
2	REG002	Sarah	Fernando	sarah@example.com	723456789
1	REG001	John	Perera	john@example.com	712345678

Figure 3: CSV file containing the required column headers

Evidence 04 – Exported Student Records

Related Acceptance Criteria: AC-005, AC-006

Description: Insert a screenshot showing multiple exported student records and confirming that the student information appears under the correct columns.

Student ID	Registration Number	First Name	Last Name	Email	Phone Number
6	REG004	kumar	hera	kumar@gmail.com	771234567
2	REG002	Sarah	Fernando	sarah@example.com	723456789
1	REG001	John	Perera	john@example.com	712345678

Figure 4: Exported student records in CSV format

Evidence 05 – Responsive Dashboard

Related Acceptance Criteria: AC-009

Description: Insert a screenshot showing the Export CSV and Add Student buttons displayed correctly on a smaller screen.

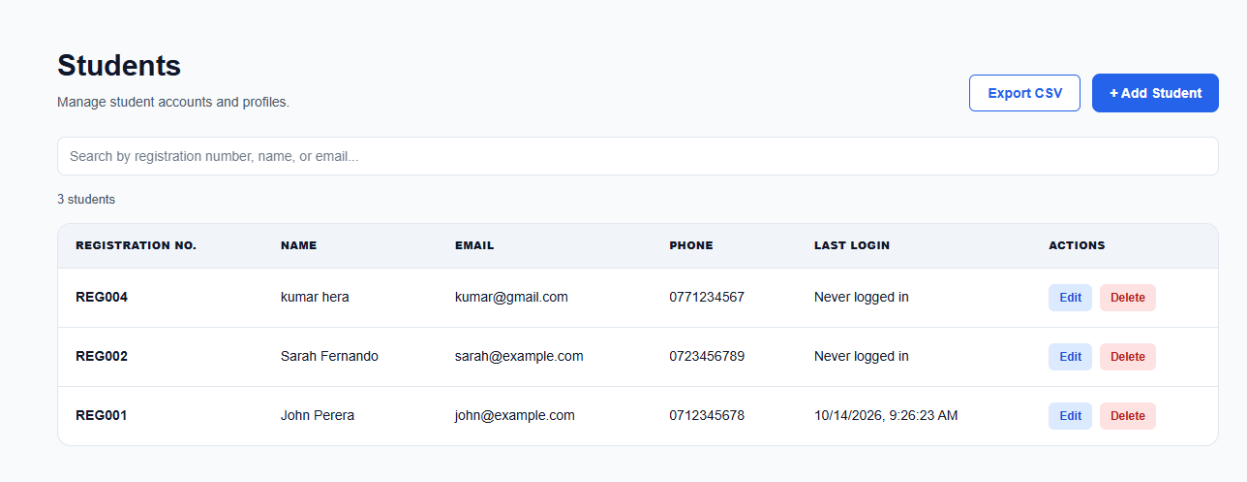


Figure 7: Responsive student management interface

6. Final Result

Item	Result
Requirements	Met
Implementation	Completed
Testing	Passed
Acceptance Criteria	All Met
Theme Persistence	Verified
Existing Functionality	Verified
Final Status	Accepted

6.1 Final Summary

CR-015 introduces the Student Records Export to CSV feature into the Student Management System.

The implementation adds an Export CSV button to the administrator's student management dashboard. Administrators can use this feature to download student records into a CSV file named students.csv.

The exported file contains Student ID, Registration Number, First Name, Last Name, Email, and Phone Number. The implementation uses the student data already retrieved from the existing backend API and generates the downloadable file directly in the browser.

The frontend export function handles CSV headers, record conversion, quotation marks, special characters, empty values, and file downloading. The dashboard stylesheet provides suitable button styling, hover effects, disabled states, and responsive layout behaviour.

The existing getStudents() controller, getAllStudents() model function, and administrator-only student route are reused without introducing another backend endpoint. No database schema changes are required.

The implementation preserves the existing login, authorization, student search, Add Student, Edit Student, and Delete Student functionality.

Based on the recorded test results, all 26 test cases passed, with a reported pass rate of 100%. The acceptance criteria are recorded as met, and the change is considered completed.

6.2 Issues / Limitations

1. The CSV export depends on the successful retrieval of student records from the existing backend API.
2. The export button is disabled while student data is loading or when the data request has failed.
3. If no student records are available, the system displays an appropriate message.
4. The export includes the records currently loaded in the frontend state. If server-side pagination is introduced later, additional handling may be required to export all records across pages.
5. The exported file contains student personal information, so administrators should store and share it securely.
6. CSV escaping helps preserve field structure, but spreadsheet applications may interpret certain values as formulas. Formula injection protection should be maintained and tested for untrusted text values.
7. The feature generates CSV files only. PDF and Excel export formats are outside the scope of CR-015.
8. The existing backend authentication and administrator authorization must remain enabled.
9. The implementation does not update or delete student information in the database.
10. No database migration is required for this change.
11. Screenshots and test evidence should be inserted into the report based on the actual development environment.
12. The final test counts and acceptance status should accurately reflect the tests executed.
13.

Student Name: _____A.Paheershana_____

Student ID: _____JF/ICT/24/04_____

Date: 15 October 2026

Signature: _____A.Paheershana_____

~ END OF REPORT ~